NEYTON Vincent
PIMOR Nicolas
PITRAT Clément

Institut Polytechnique des Sciences Avancées

Projet d'Automatique : Modélisation du comportement dynamique d'un drone

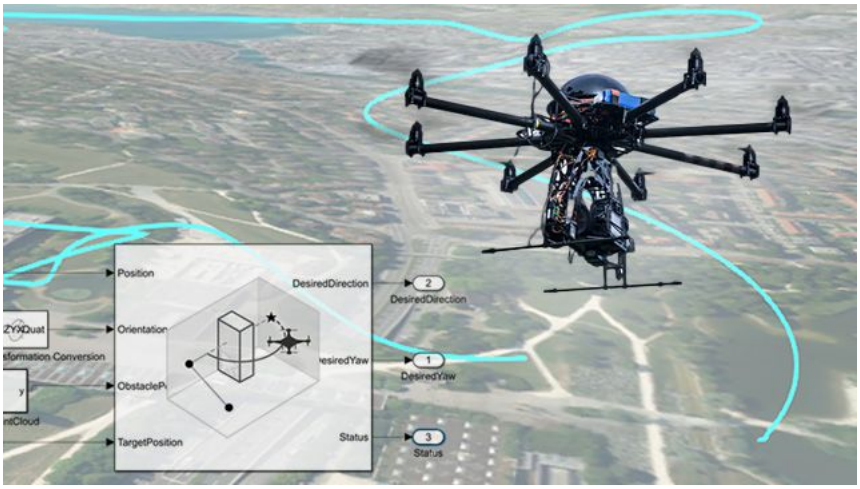


Figure 1 – Illustration d'un drone autonome en suivi de trajectoire simulé

Classe : 4TS2 / 4TS1
Professeur : A.Jacob
Date de rendu : Dimanche 24 mai 2026

1 Introduction

Ce projet vise à modéliser et commander un quadrirotor commercial de type DJI Phantom 4 dans l'environnement 3D *US City Block* sous Matlab/Simulink. L'objectif est de concevoir une loi de commande permettant au drone de rejoindre une pose cible, puis de suivre automatiquement une succession de points de passage dans un contexte de course autonome.

Un quadrirotor possède quatre actionneurs, correspondant aux quatre moteurs, pour un mouvement dans l'espace à six degrés de liberté. Le système est donc sous-actionné : il n'est pas possible de commander directement toutes les composantes du mouvement. Le choix retenu consiste à piloter une pose réduite dans le repère terrestre, composée de la position (x, y, z) et de l'angle de lacet ψ. Les déplacements horizontaux sont obtenus indirectement par l'inclinaison du drone en roulis ϕ et en tangage θ, ce qui traduit le couplage entre translation et rotation.
Cette relation entre attitude et déplacement justifie l'utilisation d'un modèle d'état pour décrire la dynamique globale du système. Le modèle utilisé repose sur une représentation d'état linéarisée autour du vol stationnaire. Le drone est assimilé à un solide rigide symétrique, ce qui conduit à une matrice d'inertie diagonale dans le repère lié au drone. La structure en croix permet de relier les vitesses de rotation des quatre moteurs à une poussée totale et à trois couples de roulis, tangage et lacet. La poussée et le contre-couple de chaque hélice sont supposés proportionnels au carré de la vitesse de rotation moteur.
Plusieurs hypothèses simplificatrices sont retenues. Les moteurs sont considérés comme suffisamment rapides devant la dynamique globale du drone, de sorte que leur régime transitoire est négligé. Les capteurs sont d'abord supposés idéaux et l'ensemble des états nécessaires à la commande est supposé disponible. Le changement de repère est décrit par les angles d'Euler selon la convention ZYX. Cette description reste valable dans un domaine de vol proche du vol stationnaire, avec des angles modérés et des commandes compatibles avec les limites physiques du drone.

La première partie du projet a permis de valider la modélisation du quadrirotor et de construire une première commande par placement de pôles. La deuxième partie introduit les contraintes et perturbations du système : saturations moteurs, préfiltrage de la consigne, défauts moteurs, bruit capteur et efforts extérieurs assimilés au vent. Enfin, la troisième partie applique le design retenu à une course autonome. Cette fiche technique ne détaille donc pas l'ensemble du projet, mais se concentre sur les éléments à forte valeur ajoutée : le design LQR optimisé, ses performances en course et les limites de l'approche proposée.
Nous présenterons d'abord le design LQR optimisé retenu. Celui-ci est construit à partir d'une normalisation de Brison, puis ajusté par le choix des pondérations sur les états et les commandes. Le réglage recherché doit trouver un compromis entre rapidité de convergence, précision du suivi, limitation des angles et effort moteur. Ce design est comparé aux commandes neutres précédentes afin de justifier les choix effectués.
Nous analyserons ensuite les résultats obtenus sur la course autonome dans *US City Block*. Une machine à états enchaîne les points de passage et met à jour les poses de référence. Les performances sont étudiées à partir du temps de parcours, du suivi de trajectoire, de l'énergie consommée et du respect des limites moteurs, puis comparées aux drones de référence fournis. Nous terminerons ce rapport par une discussion sur les limites du modèle et sur les améliorations possibles, notamment une meilleure prise en compte des fortes inclinaisons, des capteurs ou de la planification de trajectoire.

2 Présentation de notre design de contrôle LQR

Les recherches existantes ainsi que nos cours d'automatique ont montré que le PID est efficace en conditions nominales. Cependant, face à des perturbations extérieures importantes, nous nous tournons vers le Linear Quadratic Regulator (LQR), plus robuste. Ce régulateur optimal minimise un critère quadratique combinant performance et effort de commande, en linéarisant le système autour du point d'équilibre de vol stationnaire. Une fois les matrices Q et R réglées, le contrôleur est directement applicable sans relinéarisation. Nous possédons les contraintes suivantes :

Position	Atteindre le WP dans une sphère R = 2m et stop à 20cm
Angles maximaux	$\frac{\pi}{5}$rad
Moteurs	Ne pas dépasser [0,10500 RPM]
Performances	Arrivée avant le drone référence

Table 1 – Tableau récapitulatif des objectifs de design

Les constantes de temps définissent la hiérarchie temporelle des boucles de contrôle. Après itérations, nous retenons $T_{\text{angles}} = 0{,}9\,s$, valeur en dessous de laquelle le drone diverge en tentant d'atteindre trop rapidement sa consigne angulaire, au-delà de laquelle la réponse devient trop lente pour la course.
Le vecteur $e_{i,\max} = [0{,}2\ ;\ 0{,}2\ ;\ 0{,}1\ ;\ 10\,deg]$ fixe l'erreur résiduelle tolérée sur x, y, z, ψ en présence de perturbations. La tolérance en z est plus stricte (10 cm) car une erreur d'altitude non corrigée déstabilise également le suivi horizontal : le drone ne pouvant atteindre son checkpoint en z, il ne se stabilisera pas non plus correctement en x et y. Celle en ψ est plus large (10°) car le vent perturbe continuellement le lacet, une tolérance trop stricte provoquerait des corrections incessantes et une surconsommation énergétique.
Avec ces paramètres établis, nous appliquons la méthode de Brison pour initialiser les matrices de pondération Q et R du contrôleur LQR. Son principe consiste à construire les termes diagonaux de Q et R comme l'inverse du carré des écarts maximaux tolérés sur chaque état et chaque commande, adimensionnant ainsi le critère LQR : tous les états contribuent à la même échelle indépendamment de leurs unités physiques. Cette méthode fournit un point de départ physiquement cohérent, mais ne constitue pas un optimum en soi.
Les coefficients α_i et β_i sont ensuite appliqués sur cette base normalisée pour ajuster finement le compromis performance/effort :

$$\alpha_i = \begin{pmatrix} 5\ ;\ 1.5 \\ 5\ ;\ 1.5 \\ 4.5\ ;\ 1 \\ 6\ ;\ 1.5 \\ 7\ ;\ 2.5 \\ 1.5\ ;\ 0.5 \\ 0.2 \\ 0.2 \\ 0.5 \\ 0.3 \end{pmatrix} \begin{matrix} \leftarrow x\ ;\ \dot{x} \\ \leftarrow y\ ;\ \dot{y} \\ \leftarrow z\ ;\ \dot{z} \\ \leftarrow \phi\ ;\ p \\ \leftarrow \theta\ ;\ q \\ \leftarrow \psi\ ;\ r \\ \leftarrow \int x \\ \leftarrow \int y \\ \leftarrow \int z \\ \leftarrow \int \psi \end{matrix} \qquad \beta_i = \begin{pmatrix} 3 \\ 2 \\ 2 \\ 2 \end{pmatrix} \begin{matrix} \leftarrow T \\ \leftarrow C_\phi \\ \leftarrow C_\theta \\ \leftarrow C_\psi \end{matrix}$$

— x, y **(5 ; 1,5)** : une pénalité de 5 sur les positions horizontales constitue un bon compromis entre précision de suivi et effort de commande. Les vitesses $\dot{x}, \dot{y}$ sont moins pénalisées (1,5) car un dépassement en vitesse est moins critique qu'un dépassement en position.

— z **(4,5 ; 1)** : la pénalité en altitude est légèrement réduite pour conserver une meilleure réactivité lors des phases de montée et descente. La vitesse verticale est peu pénalisée (1) pour ne pas freiner inutilement la dynamique verticale.
— ϕ **(6 ; 1,5) et** θ **(7 ; 2,5)** : les angles de roulis et tangage sont les plus fortement pénalisés afin de rester dans le domaine de linéarisation ($\pm\pi/5$). Le tangage θ ainsi que la vitesse de tangage q sont plus pénalisés que pour le roulis et sa vitesse associée car le tangage s'est montré plus propice aux dépassements lors de nos simulations.
— ψ, r **(1,5 ; 0,5)** : le lacet étant continuellement perturbé par le vent, une pénalité trop élevée forcerait des corrections incessantes et coûteuses en énergie. Les valeurs restent donc faibles pour limiter ces oscillations.
— $\int x, \int y, \int z, \int \psi$ **(0,2 ; 0,2 ; 0,5 ; 0,3)** : les intégrateurs sont peu pénalisés pour accélérer le rejet des perturbations sans créer d'oscillations. L'intégrateur en z est légèrement plus élevé (0,5) pour éviter les instabilités de trajectoire liées à un checkpoint en altitude non atteint.
— β_i **(3 ; 2 ; 2 ; 2)** : la poussée T est davantage protégée (3) que les couples (2) pour limiter les oscillations rapides de commande, sources de surconsommation énergétique.

Afin de valider notre design, nous simulons une trajectoire où le drone part de la position (166,46 ; 0,5 ; 1) avec un cap de 0° et doit rejoindre la consigne (146,46 ; 20 ; 20) avec un cap de 90°, soit un déplacement de 20 m en arrière selon x, 19,5 m en montée selon z et 20 m selon y.

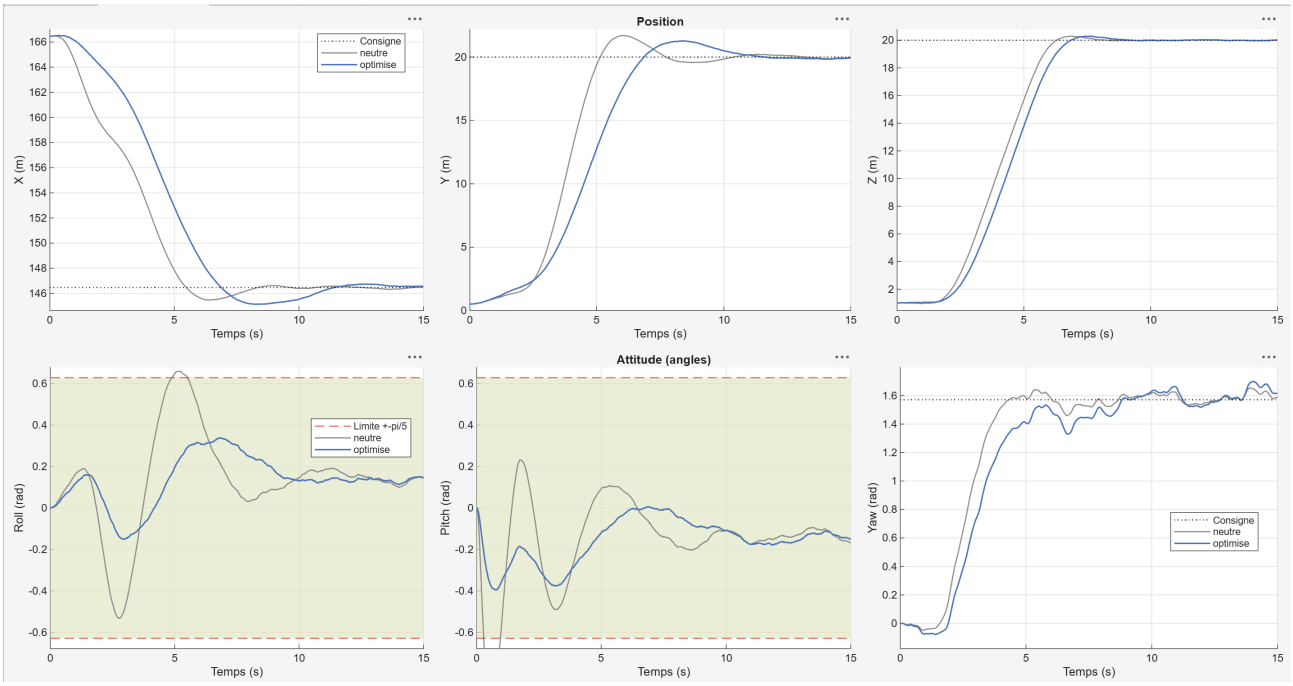


FIGURE 2 – Evolution des positions et angles au cours de la simulation

Nous pouvons remarquer que le drone optimisé atteint bien les positions et le cap escomptés, avec une légère latence. Les pénalités appliquées sur θ et ϕ sont efficaces pour maintenir les angles dans le domaine de linéarisation ($\pm\pi/5$), sans dépassement observé. [1]

1. Les résultats sont visualisables via une fenêtre MATLAB interactive s'ouvrant au cours de simulation. Il est nécessaire d'attendre la fin complète de celle-ci avant d'interagir avec la fenêtre.

3 Résultat de notre design sur la course

Après avoir établi le design que nous souhaitions dans la partie 2, nous allons désormais tester sa performance sur un enchaînement de 11 waypoints. Un waypoint sera considéré comme franchi lorsque le drone passera dans une sphère de 2 mètres de rayon autour de chaque point et de 20 centimètres pour le waypoint final. Nous nous baserons sur le drone de référence pour comparer les performances de notre drone. Evidemment, les performances sur les courses vont dépendre du vent qui est aléatoire, mais la dynamique globale restera la même. Maintenant que cela est posé, nous pouvons lancer la course et afficher le temps pour chaque waypoints et d'autres données temporelles sur la course.

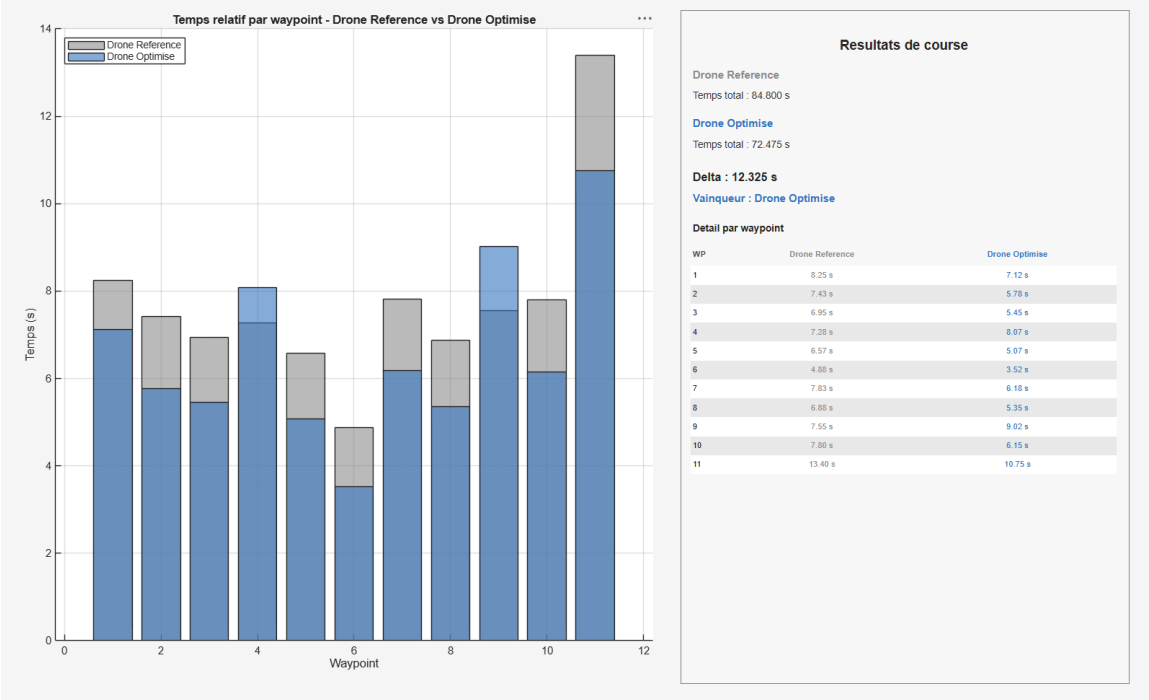


FIGURE 3 – Temps relatif entre chaque waypoint

Comme nous pouvons le voir, notre drone arrive 12.325 s avant le drone de référence. De plus, l'avance ne se crée pas seulement sur certaines phases favorables mais grâce à notre dynamique globale. Les seuls endroits où nous perdons du temps sont sur les points où la trajectoire est plus exigeante avec notamment une variation d'altitude et des espacements importants entre les points, comme sur les waypoints 4 et 9. Ces légers dépassements représentent le compromis entre rapidité de réponse et amortissements qui est un effet classique en optimisation. Cependant, par rapport au résultat global de la course, nous pouvons accepter ce compromis. De manière générale notre drone est bien plus réactif en atteignant les points de manière précise et avec une rapidité bien supérieure. Cette performance est validée par une dynamique de vol qui répond correctement en affichant les angles avec leurs limites à ne pas dépasser lors de la course qui est maintenant établie à $\frac{\pi}{5}$ rad.

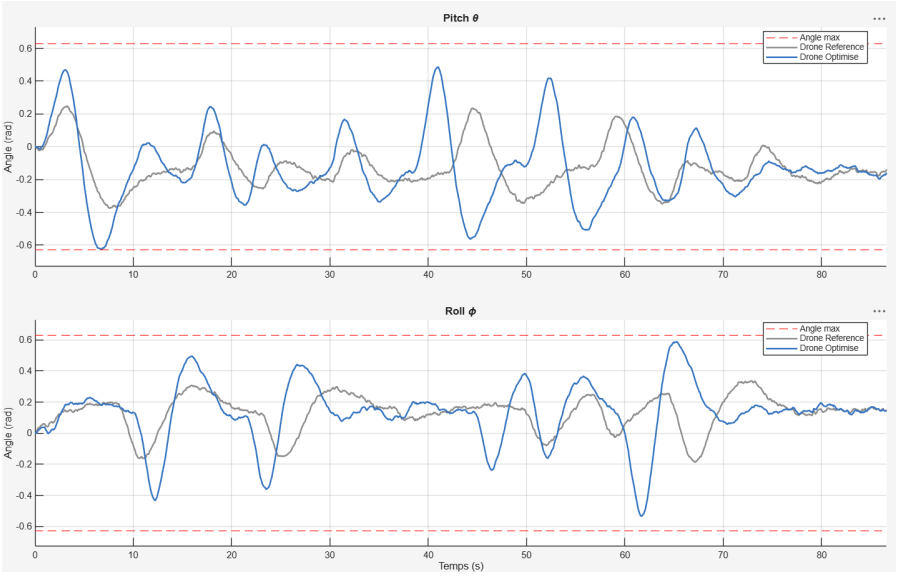


FIGURE 4 – Évolution des angles lors de la course

Nous observons que notre drone se penche bien plus que celui de référence, avec des amplitudes et des variations plus importantes et notamment en tangage ce qui contribue à notre avance. Tout cela en respectant les limites angulaires imposées ce qui renforce la pertinence de notre modèle. Il est également pertinent de noter que nous arrivons à la limite en tangage sur le premier waypoint en pitch qui montre que nous sommes arrivés à la limite du modèle imposé.
Un autre point critique de notre modèle sont les vitesses moteurs. Nous affichons, à l'instar de la partie 2, les vitesses sur chaque moteur.

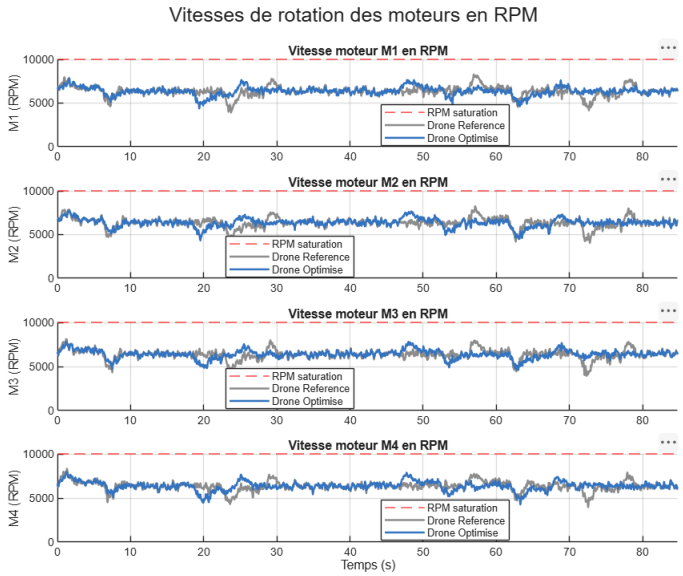


FIGURE 5 – Évolution des vitesses de rotation des moteurs lors de la course

À nouveau les saturations moteurs sont respectées et aucune anomalie n'est détectée. Nous avons même une marge de fonctionnement significative conservée, avec une répartition cohérente des vitesses sur les différents moteurs.

4 Pour aller plus loin : ajout d'un filtre passe-bas sur la consigne

Dans notre design final, la consigne de position est déjà limitée par un *Rate Limiter*. Celui-ci transforme l'échelon initial en rampe de pente bornée, ce qui permet d'éviter une sollicitation trop brutale des moteurs. Cependant, même avec cette limitation, la consigne conserve une cassure au début et à la fin de la rampe. Cette cassure correspond à une variation brutale de l'entrée en vitesse, qui peut encore exciter la dynamique du drone et favoriser certains dépassements autour des waypoints.
Pour améliorer ce point, nous nous sommes inspirés du préfiltrage présenté en cours. L'idée consiste à conserver le *Rate Limiter*, puis à ajouter un filtre passe-bas du premier ordre afin de lisser les à-coups restants. Le filtre retenu est :

$$F(p) = \frac{K_f}{1 + \tau_f p} \qquad \text{avec} \qquad K_f = 1, \quad \tau_f = 0.1 \text{ s} \tag{1}$$

Le gain statique est fixé à 1 afin de conserver la même valeur finale de consigne. La constante de temps $\tau_f = 0.1$ s reste faible devant la dynamique globale du drone : le but n'est donc pas de ralentir fortement la trajectoire de référence, mais seulement de supprimer la cassure d'entrée en vitesse.
Une attention particulière a été portée à l'initialisation du filtre. En effet, si le filtre est directement appliqué à la consigne absolue, sa sortie peut repartir de zéro au lieu de partir de la position initiale du drone. Pour éviter ce comportement, le filtrage est appliqué sur la variation de consigne, puis la condition initiale est réajoutée :

$$y_{\text{ref,filtre}}(t) = F(p)\,(y_{\text{ref}}(t) - x_{\text{init}}) + x_{\text{init}} \tag{2}$$

Cette logique a été appliquée séparément sur les quatre sorties commandées (x, y, z, ψ). Elle permet de lisser uniquement la variation demandée, sans modifier artificiellement le point de départ du drone. Le bloc de préfiltrage est donc placé directement après le *Rate Limiter*, puis le sous-système applique le même principe sur chaque composante de la consigne.

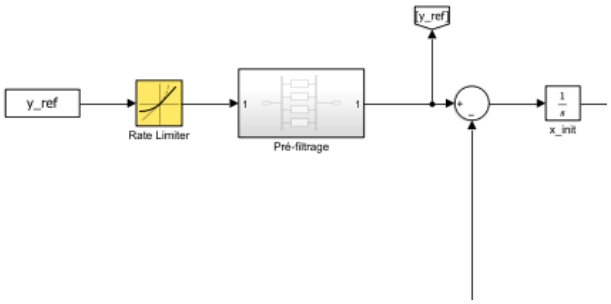


FIGURE 6 – Ajout du bloc de préfiltrage après le *Rate Limiter*

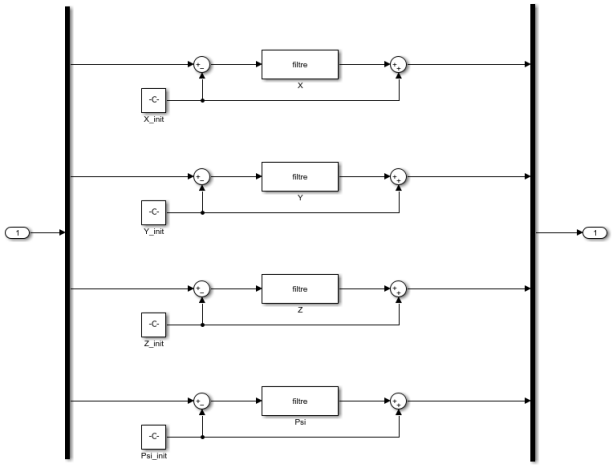


FIGURE 7 – Implémentation du filtre passe-bas sur x, y, z et ψ

La comparaison suivante illustre directement l'effet obtenu. À gauche, la consigne est seulement limitée par le *Rate Limiter* : la pente est bornée, mais les transitions restent anguleuses. À droite, l'ajout du

filtre passe-bas rend la consigne plus progressive, tout en conservant la même valeur finale. Pour rendre la comparaison plus lisible, l'axe vertical a été coupé entre 30 et 140, car cette zone ne contient pas d'information utile sur les signaux observés.

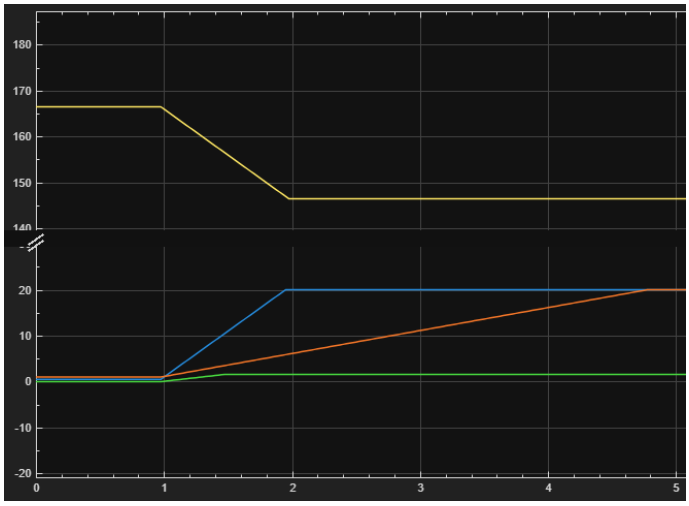


FIGURE 8 – Consigne observée sans filtre passe-bas

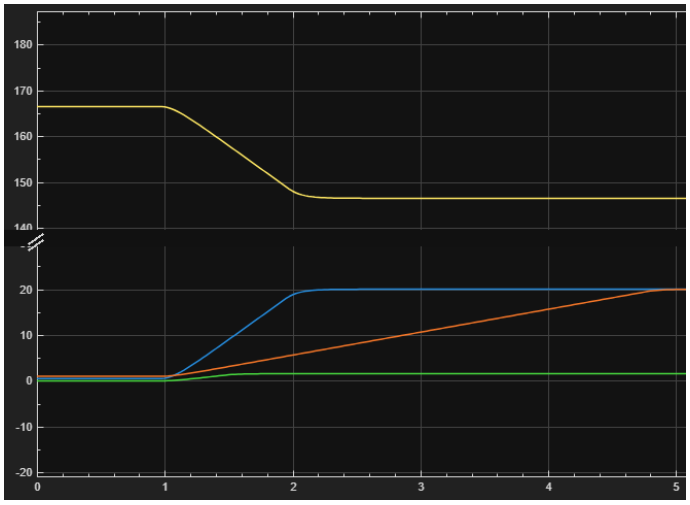


FIGURE 9 – Consigne observée avec filtre passe-bas, $\tau_f = 0.1$ s

Cette modification permet donc de réduire les à-coups envoyés à l'asservissement. En pratique, cela peut limiter les pics de commande, réduire les saturations moteurs et annuler une partie du dépassement observé lors des changements rapides de waypoint. Cette solution agit uniquement en amont de la boucle fermée : le gain de retour d'état K du LQR n'est pas modifié. Elle montre ainsi que certains dépassements ne viennent pas uniquement du réglage du LQR, mais aussi de la forme de la consigne envoyée au système.

Cependant, cette amélioration doit être interprétée avec prudence. Notre LQR final a été optimisé avec le préfiltrage initial, c'est-à-dire avec le *Rate Limiter* seul. En ajoutant ensuite un filtre passe-bas, la référence suivie par le drone devient plus douce, mais le correcteur n'a pas été redimensionné pour cette nouvelle dynamique d'entrée. Pour réellement exploiter cette amélioration, il faudrait donc refaire le design LQR en gardant ce filtre activé dès le début du réglage. Les pondérations Q et R pourraient alors être ajustées en tenant compte de la suppression de la cassure, ce qui permettrait probablement d'obtenir un compromis plus favorable entre rapidité, dépassement et saturation moteurs.

Par manque de temps, nous n'avons pas pu mener cette nouvelle optimisation. Le bloc de filtrage a donc été laissé commenté dans le modèle Simulink final. Pour l'utiliser, il suffit de le décommenter et de l'ajouter à la suite du *Rate Limiter*.

5 Conclusion et ouverture

Au terme de cette étude, nous avons pu construire une chaîne complète de modélisation, de commande et d'analyse du comportement dynamique d'un quadrirotor dans l'environnement *US City Block*. Le projet a permis de partir d'un modèle d'état linéarisé autour du vol stationnaire, puis de concevoir une commande LQR augmentée par retour intégral afin d'améliorer le suivi de consigne en présence de perturbations.

Les résultats obtenus en course valident globalement cette démarche. Notre drone termine la trajectoire avec une avance finale de 12,325 s sur le drone de référence. Cette performance montre que le design LQR retenu offre un compromis satisfaisant entre rapidité, précision et effort moteur. Les pondérations choisies permettent au drone de suivre une trajectoire cohérente tout en conservant une marge vis-à-vis des limites physiques des moteurs.

Le comportement du drone reste cependant dépendant de la géométrie des waypoints. Sur les portions simples, lorsque les changements de direction et d'altitude restent modérés, la dynamique LQR permet d'enchaîner efficacement les points de passage. En revanche, certains waypoints plus exigeants mettent en évidence les limites du réglage. C'est notamment le cas du waypoint 9, où le drone doit gagner rapidement de l'altitude tout en conservant une vitesse horizontale importante. Il peut alors dépasser la zone de validation par dessous, puis devoir freiner et revenir vers le point, ce qui entraîne une perte de temps.

Ces résultats rappellent les limites du modèle utilisé. Le drone est modélisé autour d'un vol stationnaire avec des angles supposés modérés, des moteurs considérés comme rapides et des capteurs idéaux. Ces hypothèses sont adaptées à la synthèse LQR, mais deviennent moins réalistes lorsque le drone adopte une dynamique de course plus agressive. Lorsque les angles augmentent fortement ou que les moteurs sont sollicités proche de leurs limites, le système s'éloigne du domaine de validité de la linéarisation.

Une autre limite importante concerne la représentation de l'orientation par angles d'Euler. Comme cela avait déjà été évoqué dans le rapport de l'année dernière, cette représentation peut conduire au phénomène de *gimbal lock*, lorsque deux axes de rotation deviennent alignés. Même si ce phénomène n'est pas dominant dans notre course, il souligne une faiblesse structurelle du modèle. Une amélioration naturelle serait donc de passer à une représentation par quaternions, plus robuste pour décrire des rotations importantes.

Enfin, une ouverture intéressante serait de ne plus seulement optimiser le contrôleur, mais aussi la génération des consignes. Dans notre projet, le LiDAR n'a pas été exploité dans la stratégie de course. Il pourrait pourtant être croisé avec la machine à états afin d'adapter dynamiquement le comportement du drone à la trajectoire demandée. Le drone pourrait ainsi anticiper une forte montée, réduire sa vitesse avant un virage serré ou adapter son orientation en fonction du prochain segment. Cette approche permettrait de passer d'un simple suivi de waypoints à une planification locale plus intelligente, rendant le drone plus autonome et plus performant.